Autoencoders, VAEs, and GANs

Intuition, Mathematics, and a Practical Guide to Training Each One

1. Overview

Autoencoders (AEs), Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs) all learn from unlabeled data, but they differ in what they're trying to do and how they're trained:

- **AE** — learn a compressed code for data by rebuilding it as accurately as possible. Not principled generators, but simple and stable.
- **VAE** — same rebuilding idea, but with an extra rule that turns the compressed codes into a well-organized map you can sample new points from.
- **GAN** — skip rebuilding entirely; learn to produce realistic samples by competing against a critic that's simultaneously learning to catch fakes.

This note gives the architecture, the math, a plain-language explanation of *why* the math works that way, a diagram, and — new in this version — concrete guidance for actually training and implementing each one, including the pitfalls people hit first.

2. Autoencoders (AE)

2.1 Architecture

An autoencoder is two networks glued together through a narrow middle:

- **Encoder** $f_\phi : \mathbb{R}^d \to \mathbb{R}^k$ maps input x to a latent code $z = f_\phi(x)$, with $k \ll d$ — this is the *bottleneck*.
- **Decoder** $g_\theta : \mathbb{R}^k \to \mathbb{R}^d$ maps z back to a reconstruction $\hat{x} = g_\theta(z)$.

2.2 Objective

$$\mathcal{L}_{\text{AE}}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^{N} \left\| x_i - g_\theta(f_\phi(x_i)) \right\|_2^2$$

updated jointly by gradient descent:

$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}_{\text{AE}}, \qquad \phi \leftarrow \phi - \eta \, \nabla_\phi \mathcal{L}_{\text{AE}}$$

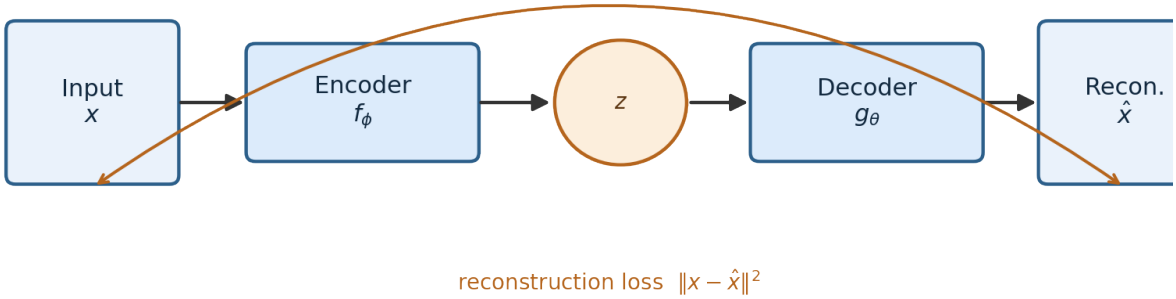


Figure 1: Autoencoder: the bottleneck forces the network to keep only what matters about x in order to rebuild it.

2.3 Qualitative picture

Think of the encoder as a lossy compressor and the decoder as the matching decompressor. The bottleneck is the whole point — if z were as big as x, the network could just copy data through and learn nothing. Squeezing z down means the network can only keep the most important, common structure; everything else gets discarded. That's *why* AEs are useful for denoising (noise isn't "important structure," so it gets thrown away for free) and anomaly detection (something unlike anything in training reconstructs badly, because the encoder never learned a good code for it).

Why AEs are poor generators. Nothing forces the latent space to be smoothly "filled in." There can be large unmapped gaps between training points' codes. Sample a random z and decode it, and you have no guarantee it lands anywhere sensible.

2.4 Training & implementation guidance

Choosing the bottleneck size k. Too large and the network can cheat toward the identity function (near-perfect reconstruction, but a useless, unstructured code). Too small and reconstructions degrade badly. Start small relative to d and increase only if reconstruction quality is clearly insufficient. Always check quality on held-out data, not just training data — an AE with enough capacity can memorize the training set even through a bottleneck if the encoder/decoder themselves are deep and expressive.

Loss and output activation. For pixel data scaled to $[0, 1]$, either: (a) sigmoid output + binary cross-entropy, treating each pixel as a Bernoulli probability, or (b) linear/tanh output + MSE. BCE tends to train a bit faster and is the more common default for image AEs.

Regularized variants worth knowing:

- *Denoising AE*: feed a corrupted $\tilde{x}$ (e.g. x + Gaussian noise or random pixel dropout) into the encoder, but compute the loss against the original clean x. This is a cheap, effective regularizer that prevents the trivial identity solution and improves robustness.
- *Sparse AE*: add an L1 penalty (or a KL penalty toward a target low activation rate) on the

latent activations to encourage a sparse code even without shrinking k.

Practical defaults: Adam optimizer, learning rate $\sim 10^{-3}$, batch size 64–256. For images, use convolutional encoder/decoder (strided convolutions to downsample, transposed convolutions or upsample+conv to go back up) rather than fully-connected layers.

Minimal training loop (PyTorch-style pseudocode):

```
for x in dataloader:
    z = encoder(x)
    x_hat = decoder(z)
    loss = mse_loss(x_hat, x)      # or bce_loss for [0,1] pixel data

    optimizer.zero_grad()
    loss.backward()                # backprop: computes gradients
    optimizer.step()                # optimizer: applies the update
```

Common pitfalls:

- Reconstruction loss keeps falling but the latent space is useless for anything downstream — expected for plain AEs; it was never optimized for latent-space structure. Use a VAE if you need that.
- Perfect training reconstruction but poor validation reconstruction usually means the bottleneck isn't actually bottlenecking — reduce k or add denoising/sparsity regularization.

3. Variational Autoencoders (VAEs)

3.1 Motivation and generative model

A VAE turns the AE into a real generative model by making the latent space probabilistic. We assume data is generated as:

$$z \sim p(z) = \mathcal{N}(0, I), \qquad x \sim p_\theta(x \mid z)$$

We'd like to maximize $p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz$, but this integral is intractable, so a VAE introduces an **approximate posterior** $q_\phi(z \mid x)$ — the encoder — to make training tractable.

3.2 The Evidence Lower Bound (ELBO)

For any $q_\phi(z \mid x)$:

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x, z)}{q_\phi(z \mid x)}\right]}_{\text{ELBO}} + D_{KL}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x))$$

Since KL divergence is always ≥ 0, the ELBO lower-bounds $\log p_\theta(x)$, and maximizing it is a tractable proxy for maximizing true likelihood. It splits into two interpretable pieces:

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction term}} - \underbrace{D_{KL}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{regularization term}}$$

Training minimizes the negative ELBO. With $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}(\sigma^2_\phi(x)))$ and prior $\mathcal{N}(0, I)$, the KL term has a closed form:

$$D_{KL} = \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right)$$

and if $p_\theta(x \mid z)$ is Gaussian with fixed variance, the reconstruction term reduces to the familiar $\|x - g_\theta(z)\|_2^2$ — so a VAE loss looks like an AE loss plus a KL penalty that pulls the latent distribution toward $\mathcal{N}(0, I)$.

3.3 The reparameterization trick

Sampling $z \sim q_\phi(z \mid x)$ directly isn't differentiable w.r.t. ϕ. The fix rewrites the sample as a deterministic function of ϕ plus independent noise:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

Now gradients can flow through z back into μ_ϕ, σ_ϕ, so ordinary backprop + an optimizer step trains everything jointly.

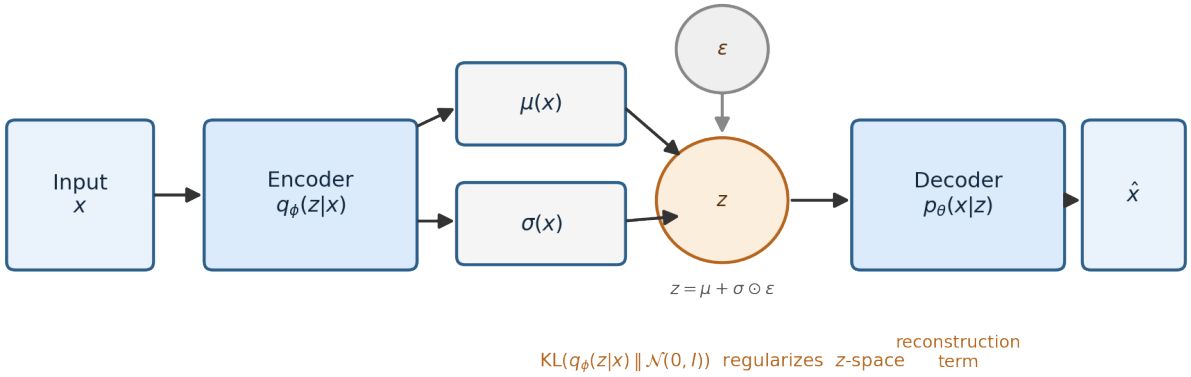


Figure 2: VAE: the encoder outputs distribution parameters μ, σ instead of one point; a sample z is drawn via the reparameterization trick, decoded, and the loss both reconstructs x and pulls $q_\phi(z|x)$ toward the prior.

3.4 Qualitative picture

The encoder no longer outputs one fixed point z — it outputs a small "cloud" (a Gaussian, described by a mean and spread) that z is randomly drawn from. Two forces act on that cloud: the reconstruction term wants it positioned wherever makes decoding x accurate; the KL term constantly pulls every cloud back toward a shared standard-normal shape centered at the origin. The compromise is a latent space where every region near the origin has been "used" by some training point's cloud, so nearby points overlap and blend smoothly — no unmapped gaps the way a plain

AE can have. That's what makes the space samplable: draw any $z \sim \mathcal{N}(0, I)$ and it lands inside some blend of training clouds rather than a dead zone.

Why VAE samples are still blurry. The decoder is still scored against a single target x using something like squared error. When it's uncertain about fine detail, the loss-minimizing move is to output the *average* of plausible options — averaging two sharp but different edges gives a soft edge. The KL term shapes the latent space; it doesn't fix this.

3.5 Training & implementation guidance

Encoder output. The encoder head outputs two vectors, `mu` and `logvar` (log-variance, not variance directly — this keeps the network from ever having to predict a negative variance and improves numerical stability). Recover the standard deviation as `std = exp(0.5 * logvar)`.

Balancing reconstruction vs. KL — the single most common problem. If the KL term dominates too early in training, the encoder collapses to just outputting the prior ($\mu \approx 0$, $\sigma \approx 1$) regardless of x — this is **posterior collapse**, and it means the latent code carries no information about the input at all. The standard fix is **KL annealing** (a "warm-up"): multiply the KL term by a weight β that ramps linearly from 0 to 1 over the first portion of training, so the model first learns to reconstruct well before the regularizer kicks in at full strength.

β-VAE. Deliberately using $\beta > 1$ overweights the KL term and tends to produce a more *disentangled* latent space (individual latent dimensions correspond to more interpretable factors of variation), at the cost of blurrier reconstructions. This is a real, adjustable trade-off, not a bug — pick β based on whether you care more about disentanglement or reconstruction fidelity.

Monitoring training properly. Log the reconstruction term and the KL term *separately*, not just their sum — a falling total loss can hide a collapsed KL term. Also track **active units**: the number of latent dimensions j where the average σ_j^2 across a batch is meaningfully below 1 (i.e. the encoder is actually using that dimension to encode information, not just passing the prior through).

Practical defaults: Adam, learning rate $\sim 10^{-3}$, latent dimension sized to the complexity of the data (tens for MNIST-scale problems, hundreds for more complex image data). Periodically sample $z \sim \mathcal{N}(0, I)$ and decode it (not just reconstruct training data) to check the model actually generates sensibly, not only reconstructs.

Minimal training loop:

```
for x in dataloader:
    mu, logvar = encoder(x)
    std = torch.exp(0.5 * logvar)
    z = mu + std * torch.randn_like(std)      # reparameterization trick
    x_hat = decoder(z)

    recon_loss = mse_loss(x_hat, x, reduction='sum')
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    loss = recon_loss + beta * kl_loss         # beta ramped 0 -> 1 during warm-up

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Common pitfalls:

- KL term goes to ~0 early and stays there → posterior collapse; add/lengthen KL annealing, or reduce decoder capacity (an overly powerful decoder can learn to ignore z entirely and reconstruct from bias terms alone on some datasets).
- Samples are consistently blurry → expected behavior of the Gaussian/MSE reconstruction term, not a bug to chase; if sharpness matters more than a structured latent space, a GAN is the better tool.

4. Generative Adversarial Networks (GANs)

4.1 Architecture

- **Generator** $G_\theta : \mathbb{R}^k \to \mathbb{R}^d$ maps noise $z \sim p(z)$ to a fake sample $G_\theta(z)$.
- **Discriminator** $D_\psi : \mathbb{R}^d \to [0, 1]$ estimates the probability a sample is real.

There's no encoder and no reconstruction target — the generator never sees real data directly, only the gradient signal passed back through D_ψ.

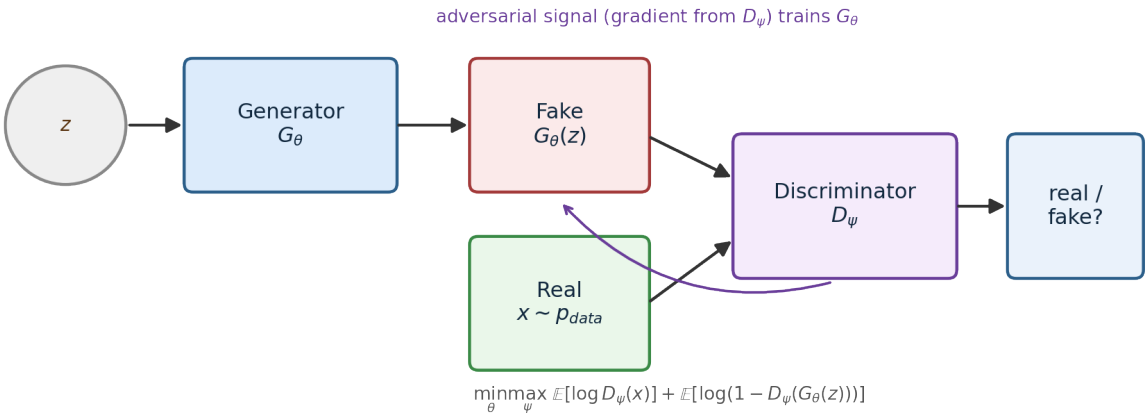


Figure 3: GAN: the generator turns noise into fake samples; the discriminator scores real vs. fake; the gradient of the discriminator's mistake is what trains the generator.

4.2 The minimax objective

$$\min_\theta \max_\psi \; V(D_\psi, G_\theta) = \mathbb{E}_{x \sim p_{\text{data}}}\big[\log D_\psi(x)\big] + \mathbb{E}_{z \sim p(z)}\big[\log\big(1 - D_\psi(G_\theta(z))\big)\big]$$

Training alternates a discriminator ascent step and a generator descent step. Early on, D_ψ easily rejects fakes, so $\log(1 - D_\psi(G_\theta(z)))$ saturates and its gradient w.r.t. θ vanishes. In practice the generator instead maximizes the **non-saturating loss**:

$$\mathcal{L}_G(\theta) = \mathbb{E}_{z \sim p(z)}\big[\log D_\psi(G_\theta(z))\big]$$

which gives much stronger gradients while the generator is still weak.

4.3 What the game converges to

For a fixed G, the optimal discriminator is:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

Substituting back shows the generator's objective, at the optimal discriminator, is equivalent to minimizing the Jensen-Shannon divergence between real and generated distributions:

$$V(D^*, G) = 2\, D_{JS}(p_{\text{data}} \,\|\, p_g) - \log 4$$

At the global optimum, $p_g = p_{\text{data}}$ and $D^*(x) = \frac{1}{2}$ everywhere — the discriminator can no longer tell real from fake.

4.4 Qualitative picture

Picture a counterfeiter (the generator) versus a forgery inspector (the discriminator). Early on, the counterfeiter is bad and the inspector easily spots fakes; that mistake is fed backward through the generator as a training signal — "this specific thing gave you away, adjust." Over time the counterfeiter improves, forcing the inspector to get more discerning, forcing the counterfeiter to improve further. Neither network has a fixed target to match — each only ever reacts to the other's current skill level. That's what makes this a genuine dynamic competition rather than a single descent, and it's the direct cause of GAN training instability: if the discriminator gets too far ahead, its gradient to the generator vanishes; if the generator finds one trick that reliably fools the current discriminator, it can collapse onto producing only that one kind of output — **mode collapse**.

4.5 Training & implementation guidance

GANs are the trickiest of the three to get training reliably, so this section is deliberately the longest.

Architecture guidelines (the "DCGAN recipe," still a strong default):

- Discriminator: strided convolutions to downsample (no pooling layers), LeakyReLU (slope ~0.2) activations, batch norm on all but the input layer.
- Generator: transposed convolutions (or upsample + conv) to upsample, ReLU activations, batch norm on all but the output layer, `tanh` output (scale real data to $[-1, 1]$ to match).
- Avoid fully-connected hidden layers for image data; convolutional architectures train far more stably.

Optimizer settings that matter more than usual. Adam with $\beta_1 = 0.5$ (not the default 0.9 — high momentum destabilizes GAN training) and $\beta_2 = 0.999$, learning rate around 2×10^{-4} for both networks as a starting point.

The alternating training loop:

```
for x_real in dataloader:
    # 1. Discriminator step
    z = torch.randn(batch_size, latent_dim)
    x_fake = generator(z).detach()          # detach: don't train G here
```

```
    d_loss = -(torch.log(discriminator(x_real)).mean()
               + torch.log(1 - discriminator(x_fake)).mean())
    d_optimizer.zero_grad()
    d_loss.backward()
    d_optimizer.step()

    # 2. Generator step (non-saturating loss)
    z = torch.randn(batch_size, latent_dim)
    x_fake = generator(z)
    g_loss = -torch.log(discriminator(x_fake)).mean()
    g_optimizer.zero_grad()
    g_loss.backward()
    g_optimizer.step()
```

Stabilization techniques worth knowing, roughly in order of how often you'll reach for them:

- *Non-saturating generator loss* (above) — essentially mandatory; the original minimax generator loss is rarely used in practice.
- *Label smoothing* — train the discriminator against soft real labels (e.g. 0.9 instead of 1.0) so it never becomes overconfident, which keeps its gradients more useful to the generator.
- *Separate real/fake batches* through batch norm rather than concatenating them — prevents batch statistics from mixing real and fake samples in a way that leaks information and destabilizes training.
- *Two Time-Scale Update Rule (TTUR)* — use a different (often lower) learning rate for the generator than the discriminator, since a discriminator that's slightly ahead is more useful than one that's behind.
- *Spectral normalization* on discriminator weights — constrains the discriminator's Lipschitz constant, directly improving training stability.
- *Wasserstein GAN (WGAN / WGAN-GP)* — replaces the JS-divergence objective with an Earth-Mover distance and a "critic" (unbounded output, no sigmoid) that must satisfy a Lipschitz constraint, enforced via a gradient penalty. This is the most effective known fix for both vanishing gradients and mode collapse, at the cost of a more involved training loop.

Monitoring training — the part that surprises people coming from AEs/VAEs. Unlike a reconstruction loss, D and G loss values going down do **not** straightforwardly mean "the model is getting better" — because each loss is relative to an opponent that is itself changing. Two practical consequences:

- Watch for the two losses to roughly *stabilize relative to each other* (neither collapsing to ~0 nor diverging to infinity), rather than expecting either to monotonically decrease.
- You still need to *look at samples* periodically during training, and ideally track a quantitative image-quality metric like **FID (Fréchet Inception Distance)** — lower is better — since loss curves alone can look fine while the model has quietly mode-collapsed.

Common pitfalls:

- Generator loss shooting to very large values, discriminator loss to ~0 → discriminator has "won" and stopped providing useful gradient; lower its learning rate relative to the generator's, add label smoothing, or add spectral normalization.

- All generated samples look nearly identical → mode collapse; try minibatch discrimination, unrolled GANs, or switch to a WGAN-GP objective.
- Loss curves look stable but samples are visibly poor → this is normal for GANs; trust FID and visual inspection over the loss numbers.

5. Shared Training Mechanics

Regardless of architecture, every gradient-based update here follows the same two stages:

1. **Backpropagation** computes $\nabla_\theta \mathcal{L}$ for every parameter via the chain rule. It only computes gradients — it never modifies weights itself.
2. **Optimizer step** consumes those gradients to update parameters, e.g. vanilla SGD ($\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$) or an adaptive method like Adam that rescales the step using running estimates of gradient moments.

For GANs this two-stage update is applied *twice* per iteration — once for ψ, once for θ — each with the other network's parameters held fixed, which is exactly what makes GAN training a coupled system rather than a single optimization.

6. Qualitative Summary: Three Ways to Think About It

- **AE — "compress and rebuild."** Graded only on how close the rebuild is to the original. No requirement that the compressed codes form a usable map.
- **VAE — "compress into a well-organized map, then rebuild."** Same rebuilding task, plus a rule that your codes must collectively look like they were drawn from a standard normal cloud — which is what makes the space samplable afterward.
- **GAN — "fool a critic, with no rebuilding at all."** Never shown a target to match; only ever told, after the fact, whether the output looked real to an opponent simultaneously getting better at telling the difference. No pointwise target to average over, so outputs stay sharp — but no direct, stable signal either, so training is harder to control.

7. Comparison Table

	Autoencoder (AE)	VAE	GAN
Networks	Encoder + Decoder	Encoder + Decoder	Generator + Discriminator
Loss type	Reconstruction (direct)	Reconstruction + KL (ELBO)	Adversarial (minimax)
Needs paired input/output	Yes	Yes	No — generator only needs noise
Latent space	Unstructured	Structured, $\mathcal{N}(0, I)$ prior	Implicit, not directly regularized
Can sample novel data	Not principled	Yes, by construction	Yes, by construction
Typical output sharpness	Blurry	Blurry–moderate	Sharp
Training stability	Stable	Stable	Unstable (needs active stabilization)

	Autoencoder (AE)	VAE	GAN
Hardest practical failure mode	Trivial identity mapping	Posterior collapse	Mode collapse
Core mathematical tool	Pointwise loss minimization	Variational inference (ELBO)	Game theory (minimax, JS divergence)

8. Which One Should You Reach For?

- Need compression, denoising, or an anomaly-detection score, and don't care about generating novel samples → **AE**.
- Need to sample new data *and* want a structured, interpretable, or interpolatable latent space, and can tolerate some blurriness → **VAE**.
- Need the sharpest possible samples, don't need an explicit latent-space structure, and are prepared to spend time on training stability → **GAN**.

9. Summary

- **AEs** learn reconstruction through a bottleneck — simple and stable, but the latent space has no guaranteed structure and the model isn't a principled generator.
- **VAEs** add a probabilistic latent space and optimize the ELBO, using the reparameterization trick to stay differentiable. The main implementation fight is balancing reconstruction against the KL term (posterior collapse); get that right with KL annealing and the model becomes a genuinely samplable generator, at some cost in sharpness.
- **GANs** drop reconstruction entirely for a minimax game, equivalent at the optimal discriminator to minimizing a Jensen-Shannon divergence. Sample quality is the best of the three, but training is a coupled dynamical system rather than a single descent, so stabilization techniques (non-saturating loss, TTUR, spectral norm, WGAN-GP) aren't optional extras — they're usually required to get a GAN to train at all.